



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

Lezione 5 - Algoritmi di ricerca informata



Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata, definizione

Mentre gli algoritmi di ricerca non informata hanno l'obiettivo di trovare soluzioni senza avere a disposizione alcuna informazione aggiuntiva, gli algoritmi di ricerca informata **sfruttano conoscenze specifiche del problema**, così da essere più efficienti.

Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata, definizione

Mentre gli algoritmi di ricerca non informata hanno l'obiettivo di trovare soluzioni senza avere a disposizione alcuna informazione aggiuntiva, gli algoritmi di ricerca informata **sfruttano conoscenze specifiche del problema**, così da essere più efficienti.

L'approccio generale viene chiamato ricerca *best-first*: questo è un'istanza dell'algoritmo generale di ricerca in cui il nodo da espandere viene scelto sulla base di una *funzione di valutazione* $f(n)$.

Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata, definizione

Mentre gli algoritmi di ricerca non informata hanno l'obiettivo di trovare soluzioni senza avere a disposizione alcuna informazione aggiuntiva, gli algoritmi di ricerca informata **sfruttano conoscenze specifiche del problema**, così da essere più efficienti.

L'approccio generale viene chiamato ricerca *best-first*: questo è un'istanza dell'algoritmo generale di ricerca in cui il nodo da espandere viene scelto sulla base di una *funzione di valutazione* $f(n)$.

In generale, l'implementazione della ricerca best-first è identica a quella della ricerca a costo uniforme: l'unica differenza è che la coda a priorità è ordinata in base ad $f(n)$ piuttosto che $g(n)$.

Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata, definizione

Mentre gli algoritmi di ricerca non informata hanno l'obiettivo di trovare soluzioni senza avere a disposizione alcuna informazione aggiuntiva, gli algoritmi di ricerca informata **sfruttano conoscenze specifiche del problema**, così da essere più efficienti.

L'approccio generale viene chiamato ricerca *best-first*: questo è un'istanza dell'algoritmo generale di ricerca in cui il nodo da espandere viene scelto sulla base di una *funzione di valutazione* $f(n)$.

In generale, l'implementazione della ricerca best-first è identica a quella della ricerca a costo uniforme: l'unica differenza è che la coda a priorità è ordinata in base ad $f(n)$ piuttosto che $g(n)$.

Molti algoritmi di ricerca best-first includono quale componente di $f(n)$ una funzione euristica denominata $h(n)$.

$h(n)$. Funzione che indica il costo stimato del cammino più conveniente dallo stato del nodo n ad uno stato obiettivo.

Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata, definizione

Mentre gli algoritmi di ricerca non informata hanno l'obiettivo di trovare soluzioni senza avere a disposizione alcuna informazione aggiuntiva, gli algoritmi di ricerca informata **sfruttano conoscenze specifiche del problema**, così da essere più efficienti.

L'approccio generale viene chiamato ricerca *best-first*: questo è un'istanza dell'algoritmo generale di ricerca in cui il nodo da espandere viene scelto sulla base di una *funzione di valutazione* $f(n)$.

In generale, l'implementazione della ricerca *best-first* è identica a quella della ricerca a costo uniforme: l'unica differenza è che la coda a priorità è ordinata in base ad $f(n)$ piuttosto che $g(n)$.

Molti algoritmi di ricerca *best-first* includono quale componente di $f(n)$ una funzione euristica denominata $h(n)$.

$h(n)$. Funzione che indica il costo stimato del cammino più conveniente dallo stato del nodo n ad uno stato obiettivo.

Possiamo sostenere che le funzioni euristiche sono il modo più comune, e forse migliore, per inserire conoscenza aggiuntiva in un algoritmo di ricerca.

Algoritmi di Ricerca Informata

Algoritmi Best-First, implementazione

```
function RICERCA-COSTO-UNIFORME RICERCA-BEST-FIRST(problema, strategia)
    returns una soluzione o un fallimento

nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
frontiera ← una coda a priorità ordinata per COSTO-STIMATO, con nodo unico elemento
esplorati ← un insieme vuoto

loop do

    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati

    for each azione in problema.AZIONI(nodo.STATO) do
        figlio ← NODO-FIGLIO(problema, stato, azione)
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            frontiera ← INSERISCI(figlio, frontiera)
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
        else if figlio.STATO è in frontiera con COSTO-STIMATO più alto then
            sostituisci quel nodo frontiera con figlio
```

Algoritmi di Ricerca Informata

Algoritmi Best-First, implementazione

```
function RICERCA-COSTO-UNIFORME RICERCA-BEST-FIRST(problema, strategia)
    returns una soluzione o un fallimento
```

```
nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
```

```
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
```

```
frontiera ← una coda a priorità ordinata per COSTO-STIMATO, con nodo unico elemento
```

```
esplorati ← un insieme vuoto
```

```
loop do
```

```
    if la frontiera è vuota then return fallimento
    nodo ← POP(frontiera)
```

```
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
    aggiungi nodo.STATO a esplorati
```

```
    for each azione in problema.AZIONI(nodo.STATO) do
```

```
        figlio ← NODO-FIGLIO(problema, stato, azione)
```

```
        if figlio.STATO non è in esplorati e figlio non è in frontiera then
            frontiera ← INSERISCI(figlio, frontiera)
```

```
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
```

```
            else if figlio.STATO è in frontiera con COSTO-STIMATO più alto then
                sostituisci quel nodo frontiera con figlio
```

/* Banalmente, servirà modificare il modo in cui la coda a priorità ordina gli elementi al suo interno. Il costo stimato darà dato dalla funzione $h(n)$. */

Algoritmi Best-First, implementazione

```
function RICERCA-COSTO-UNIFORME RICERCA-BEST-FIRST(problema, strategia)  
    returns una soluzione o un fallimento
```

```
nodo ← un nodo con stato = problema.STATO-INIZIALE e COSTO-DI-CAMMINO=0
```

```
frontiera ← una coda a priorità ordinata per COSTO-CAMMINO, con nodo unico elemento
```

```
frontiera ← una coda a priorità ordinata per COSTO-STIMATO, con nodo unico elemento
```

```
esplorati ← un insieme vuoto
```

```
loop do
```

```
    if la frontiera è vuota then return fallimento
```

```
    nodo ← POP(frontiera)
```

```
    if problema.TEST-OBIETTIVO(nodo.STATO) then return SOLUZIONE(nodo)
```

```
    aggiungi nodo.STATO a esplorati
```

```
    for each azione in
```

```
        figlio ← NODO(azione, nodo)
```

```
        if figlio.STATO non è in esplorati e figlio non è in frontiera then  
            frontiera ← INSERISCI(figlio, frontiera)
```

```
        else if figlio.STATO è in frontiera con COSTO-CAMMINO più alto then
```

```
            else if figlio.STATO è in frontiera con COSTO-STIMATO più alto then  
                sostituisci quel nodo frontiera con figlio
```

/* Chiaramente, anche il modo in cui la coda sarà manipolata dovrà cambiare e sarà basata sul costo stimato. */

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l’aggettivo più appropriato.

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l’aggettivo più appropriato.

L’algoritmo cerca sempre di **espandere il nodo più vicino all’obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l'aggettivo più appropriato.

L'algoritmo cerca sempre di **espandere il nodo più vicino all'obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Di conseguenza la valutazione dei nodi viene fatta applicando direttamente la funzione euristica $f(n) = h(n)$, con $h \geq 0$.

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l'aggettivo più appropriato.

L'algoritmo cerca sempre di **espandere il nodo più vicino all'obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Di conseguenza la valutazione dei nodi viene fatta applicando direttamente la funzione euristica $f(n) = h(n)$, con $h \geq 0$.

Vediamo come funziona un algoritmo greedy riconsiderando l'esempio di navigazione da Arad a Bucarest.

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l’aggettivo più appropriato.

L’algoritmo cerca sempre di **espandere il nodo più vicino all’obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Di conseguenza la valutazione dei nodi viene fatta applicando direttamente la funzione euristica $f(n) = h(n)$, con $h \geq 0$.

Vediamo come funziona un algoritmo greedy riconsiderando l’esempio di navigazione da Arad a Bucarest.

Una delle possibili euristiche, in questo caso, consiste nel considerare *la distanza in linea d’aria tra i nodi di partenza e obiettivo*, che indicheremo con h_{DLA} .

Algoritmi di Ricerca Informata

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l’aggettivo più appropriato.

L’algoritmo cerca sempre di **espandere il nodo più vicino all’obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Di conseguenza la valutazione dei nodi viene fatta applicando direttamente la funzione euristica $f(n) = h(n)$, con $h \geq 0$.

Vediamo come funziona un algoritmo greedy riconsiderando l’esempio di navigazione da Arad a Bucarest.

Una delle possibili euristiche, in questo caso, consiste nel considerare *la distanza in linea d’aria tra i nodi di partenza e obiettivo*, che indicheremo con h_{DLA} .

Chiaramente, per poter utilizzare questa euristica dovremo conoscere a priori tutte le distanze in linea d’aria verso Bucarest —> questa è un’informazione che rende, appunto l’algoritmo di ricerca *informato*.

Algoritmi di Ricerca Informata

Ricerca Best-First Greedy

Gli algoritmi greedy sono tra i più utilizzati in pratica per problemi di ottimizzazione in diversi contesti. In italiano, il termine greedy viene tradotto come “goloso”, poiché come vedremo è esattamente questo l’aggettivo più appropriato.

L’algoritmo cerca sempre di **espandere il nodo più vicino all’obiettivo**, sulla base del fatto che è probabile che questo porti rapidamente ad una soluzione.

Di conseguenza la valutazione dei nodi viene fatta applicando direttamente la funzione euristica $f(n) = h(n)$, con $h \geq 0$.

Vediamo come funziona un algoritmo greedy riconsiderando l’esempio di navigazione da Arad a Bucarest.

Una delle possibili euristiche, in questo caso, consiste nel considerare *la distanza in linea d’aria tra i nodi di partenza e obiettivo*, che indicheremo con h_{DLA} .

Chiaramente, per poter utilizzare questa euristica dovremo conoscere a priori tutte le distanze in linea d’aria verso Bucarest —> questa è un’informazione che rende, appunto l’algoritmo di ricerca *informato*.

Da notare c’è anche il fatto che, per definire un’euristica del genere, bisogna avere consapevolezza di una correlazione tra la distanza in linea d’aria e quella su strada.

Algoritmi di Ricerca Informata

Ricerca Best-First Greedy, un esempio

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

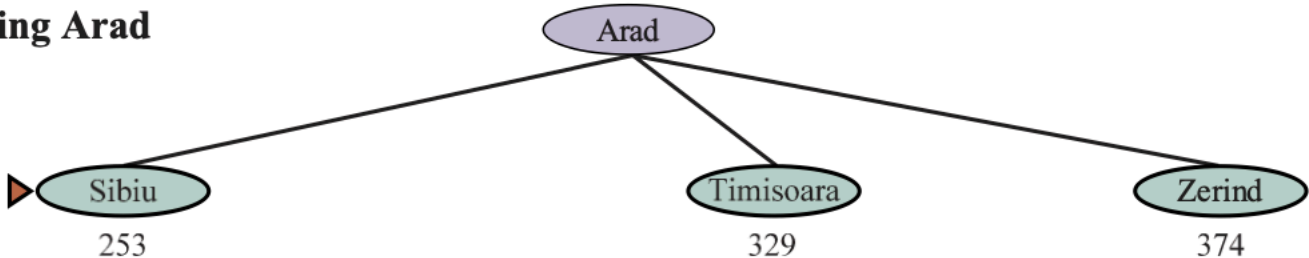
Sulla base di queste distanze, possiamo definire l'albero di ricerca che sarà utilizzato dall'algoritmo per trovare la soluzione.

Ricerca Best-First Greedy, un esempio

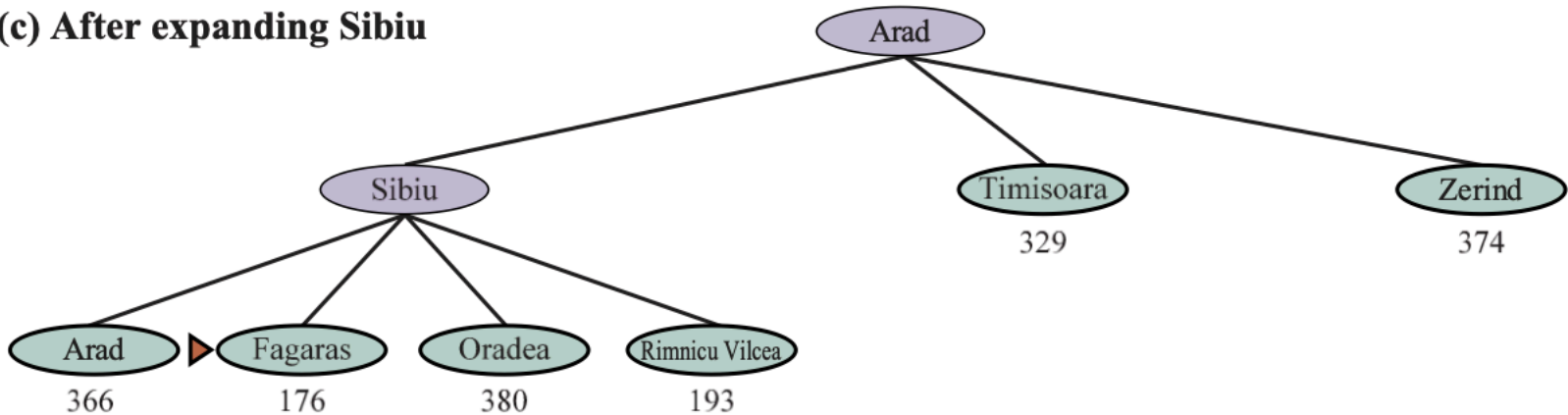
(a) The initial state



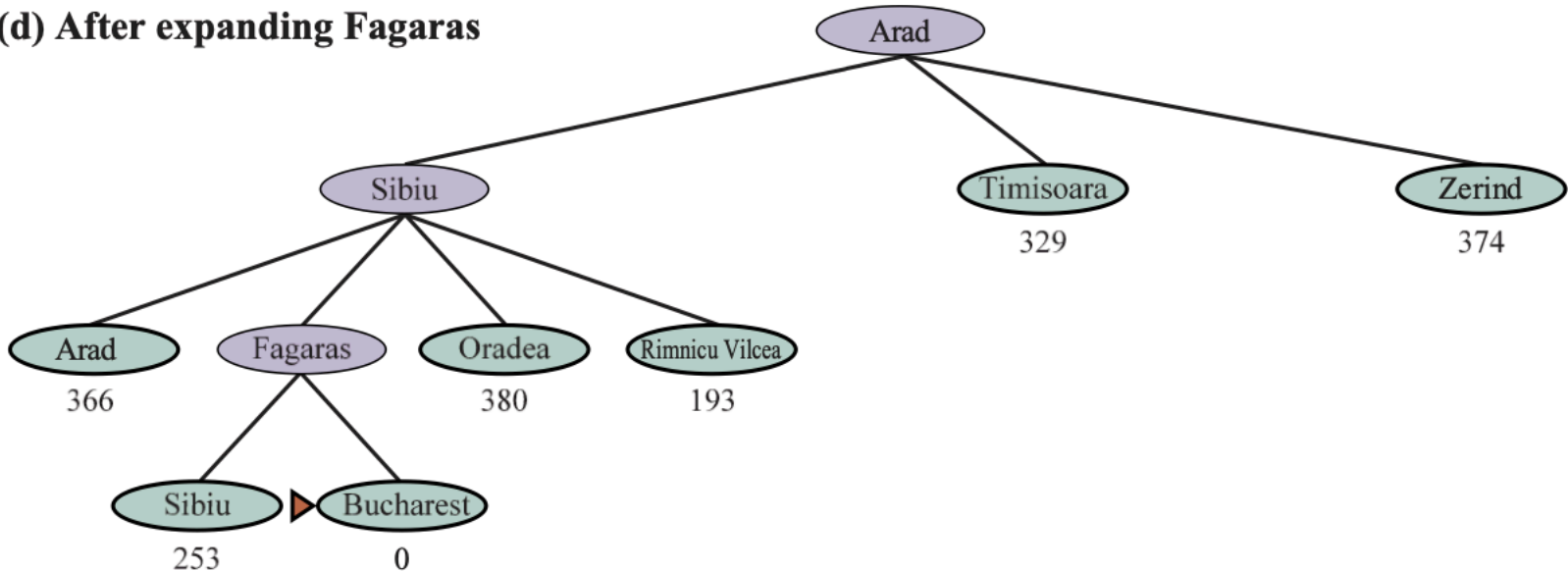
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras

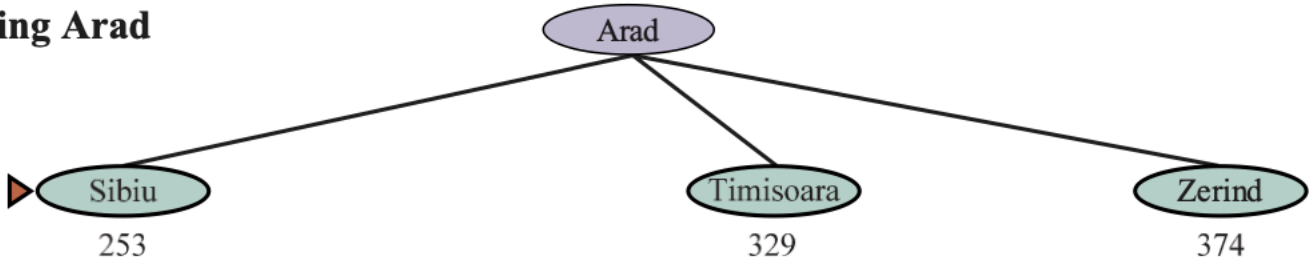


Ricerca Best-First Greedy, un esempio

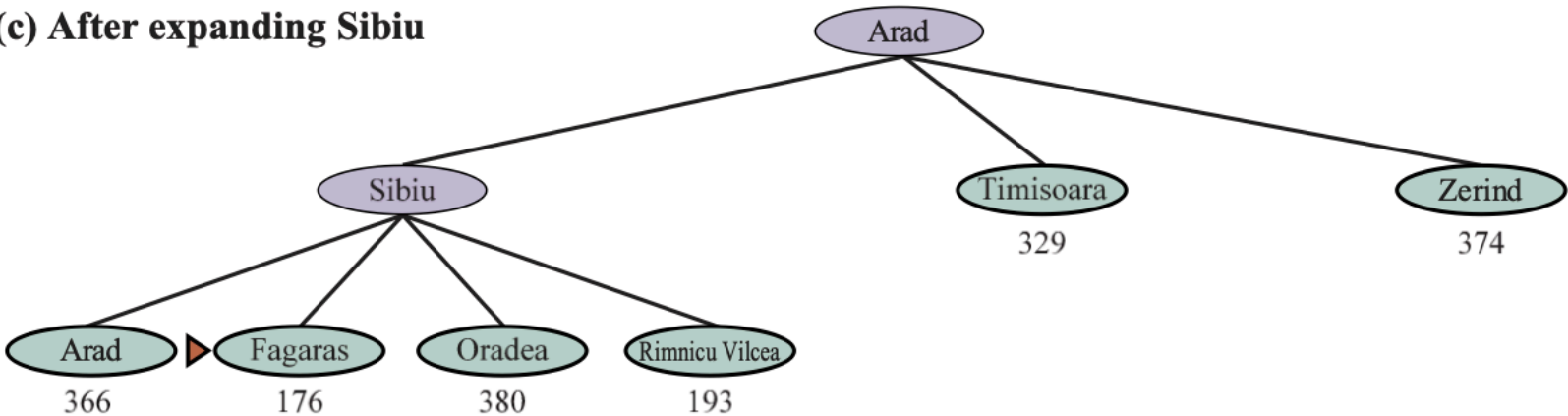
(a) The initial state



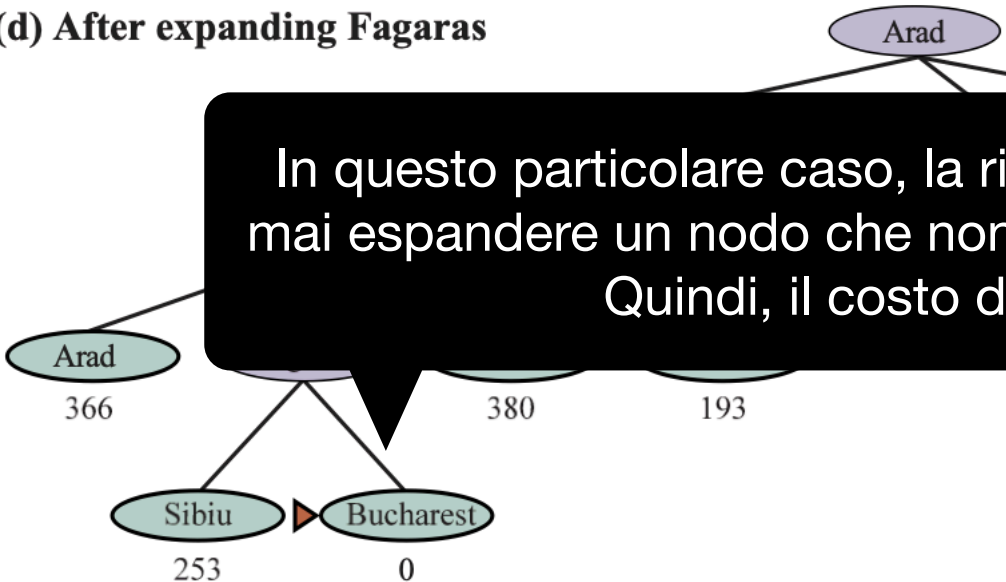
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



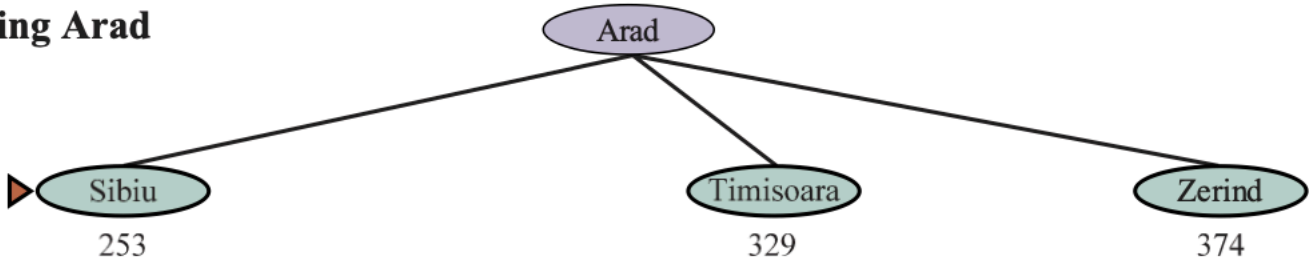
In questo particolare caso, la ricerca trova una soluzione senza mai espandere un nodo che non sia sul cammino della soluzione. Quindi, il costo di ricerca è minimo.

Ricerca Best-First Greedy, un esempio

(a) The initial state



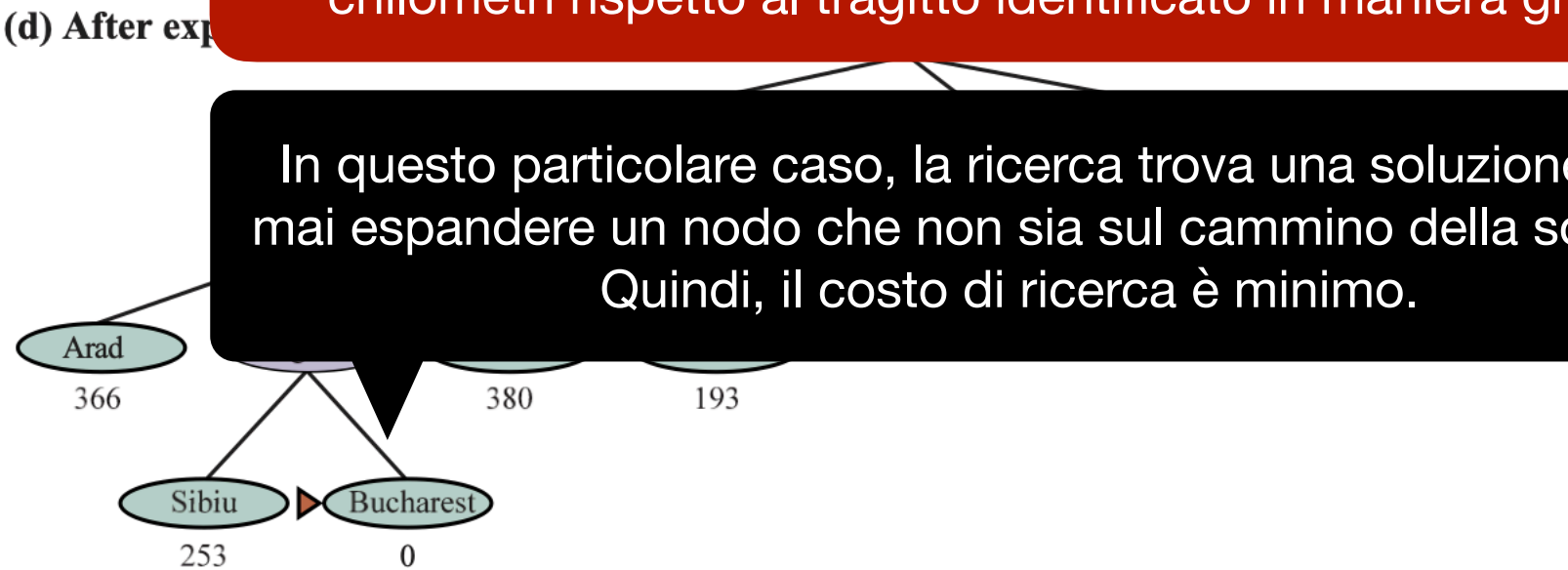
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding



Tuttavia, la soluzione non è ottima. Esiste un altro cammino, che da Sibiu va a Vilcea e poi a Pitesti che farebbe risparmiare 32 chilometri rispetto al tragitto identificato in maniera greedy.

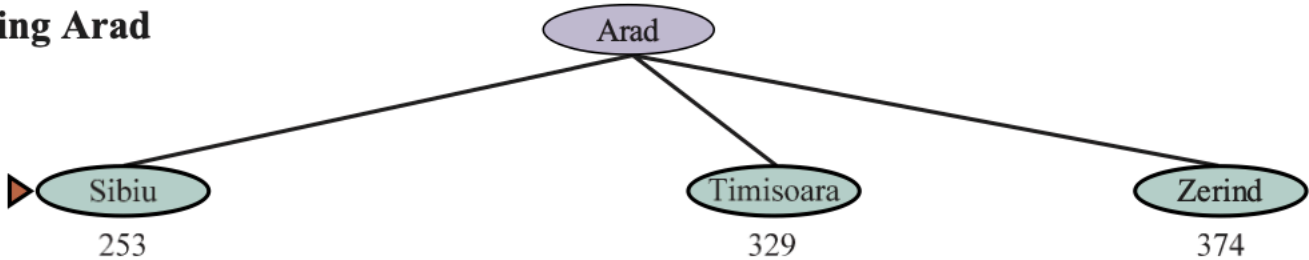
In questo particolare caso, la ricerca trova una soluzione senza mai espandere un nodo che non sia sul cammino della soluzione. Quindi, il costo di ricerca è minimo.

Ricerca Best-First Greedy, un esempio

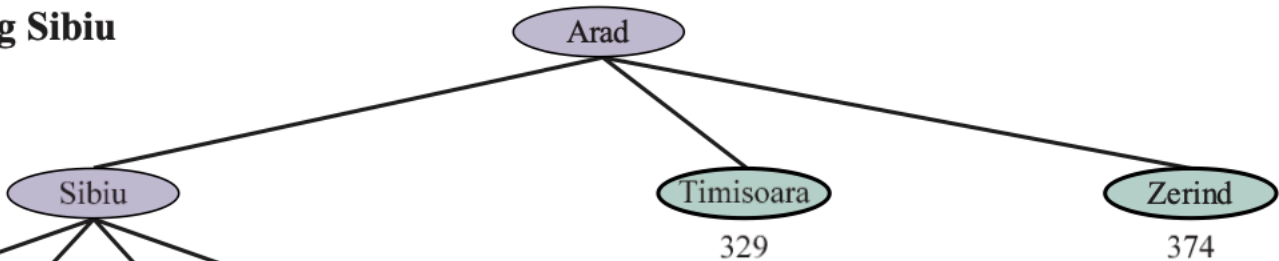
(a) The initial state



(b) After expanding Arad



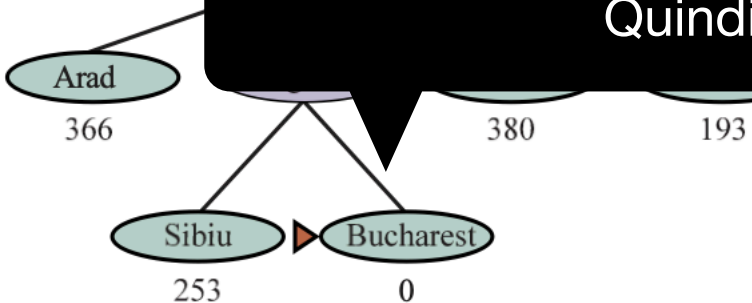
(c) After expanding Sibiu



tima. Esiste un altro cammino, che Pitesti che farebbe risparmiare 32 o identificato in maniera greedy.

Ecco perché l'algoritmo viene definito "goloso": ad ogni passo cerca sempre di arrivare più vicino all'obiettivo, non considerando cammini alternativi con stima di costo maggiore.

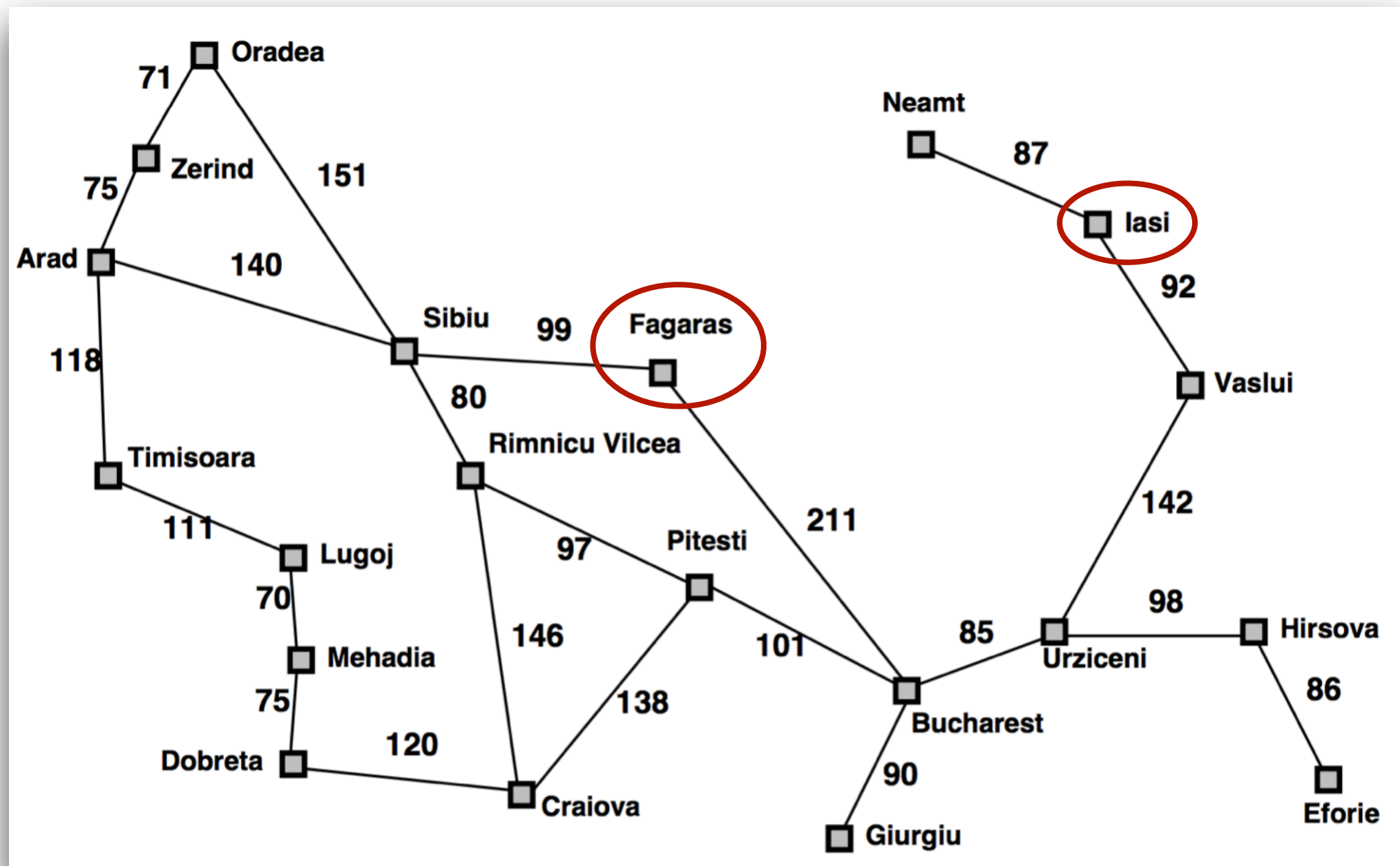
In questo particolare caso, la ricerca trova una soluzione senza mai espandere un nodo che non sia sul cammino della soluzione. Quindi, il costo di ricerca è minimo.



Algoritmi di Ricerca Informata

Ricerca Best-First Greedy, un esempio

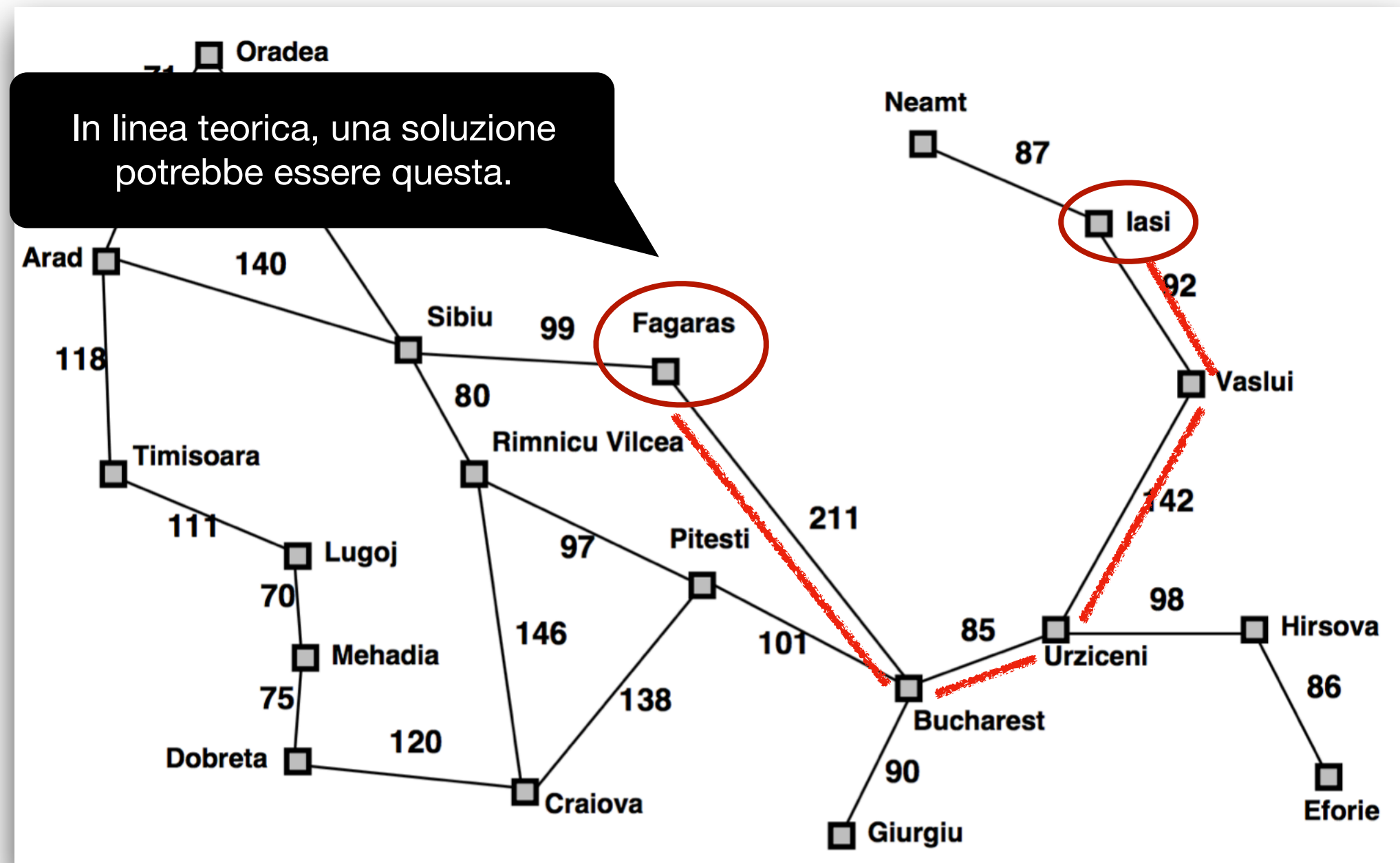
Oltre a non essere ottimo, l'algoritmo non è neanche completo.
Consideriamo il problema di andare da Iasi a Fagaras...



Algoritmi di Ricerca Informata

Ricerca Best-First Greedy, un esempio

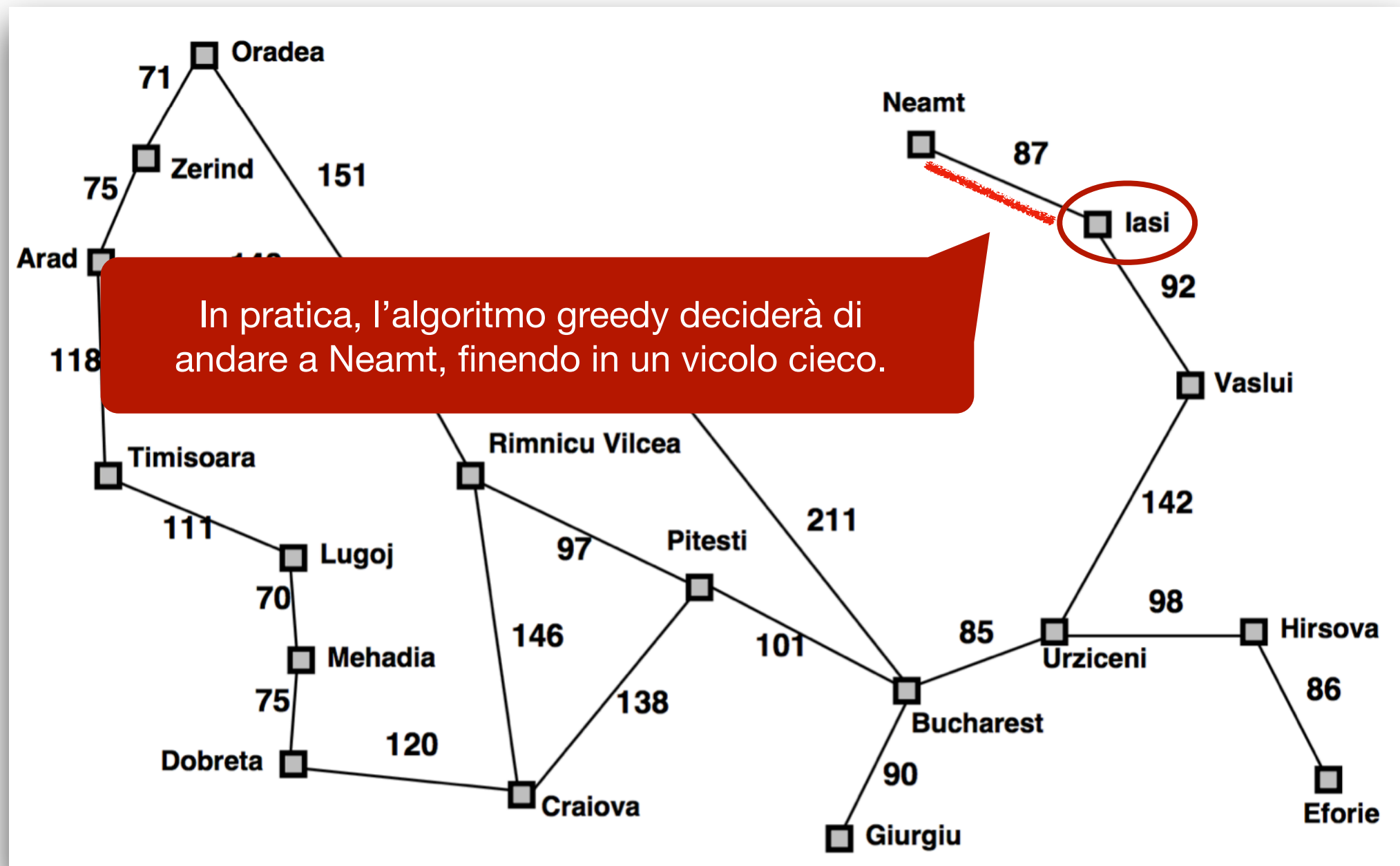
Oltre a non essere ottimo, l'algoritmo non è neanche completo.
Consideriamo il problema di andare da Iasi a Fagaras...



Algoritmi di Ricerca Informata

Ricerca Best-First Greedy, un esempio

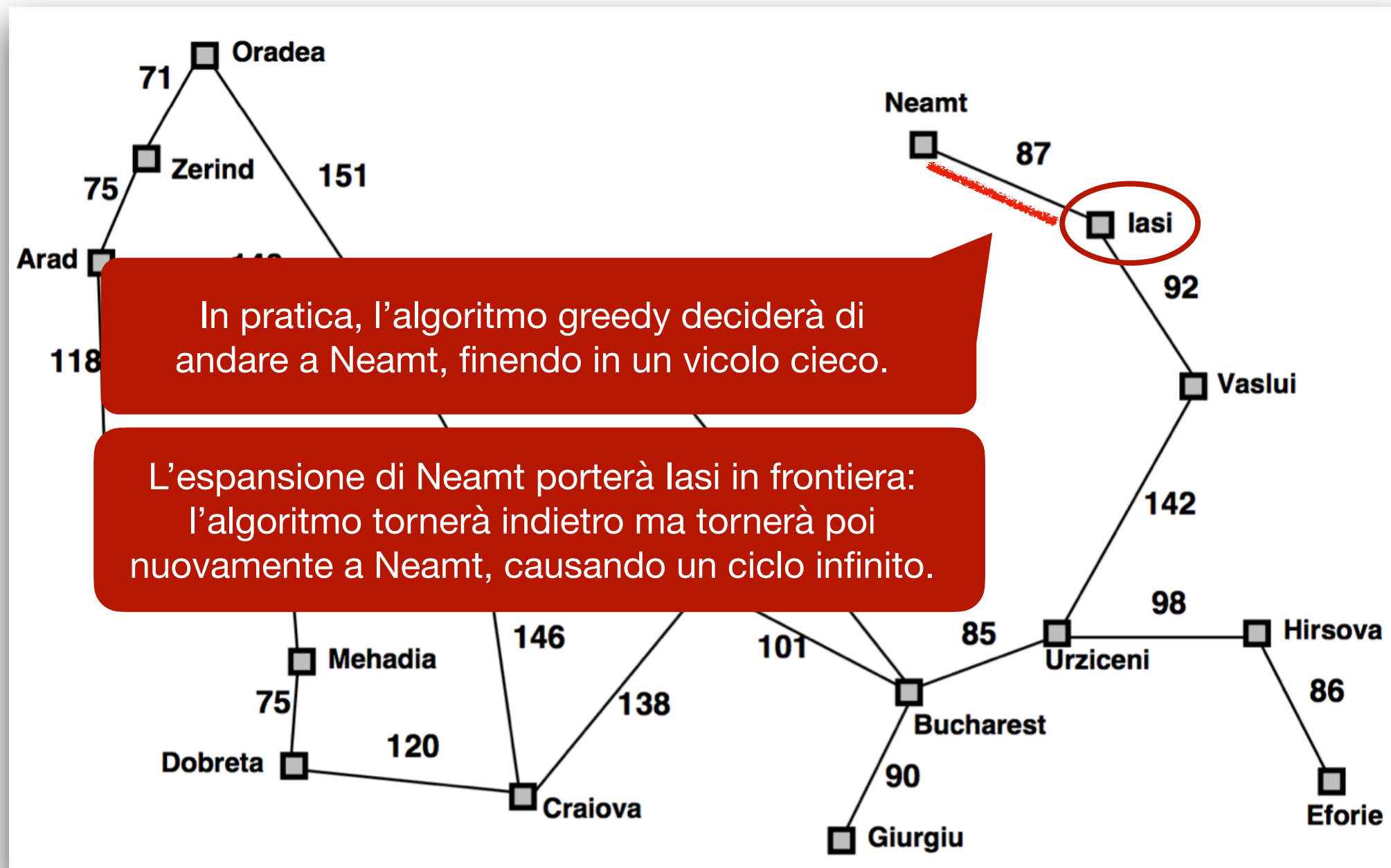
Oltre a non essere ottimo, l'algoritmo non è neanche completo.
Consideriamo il problema di andare da Iasi a Faragas...



Algoritmi di Ricerca Informata

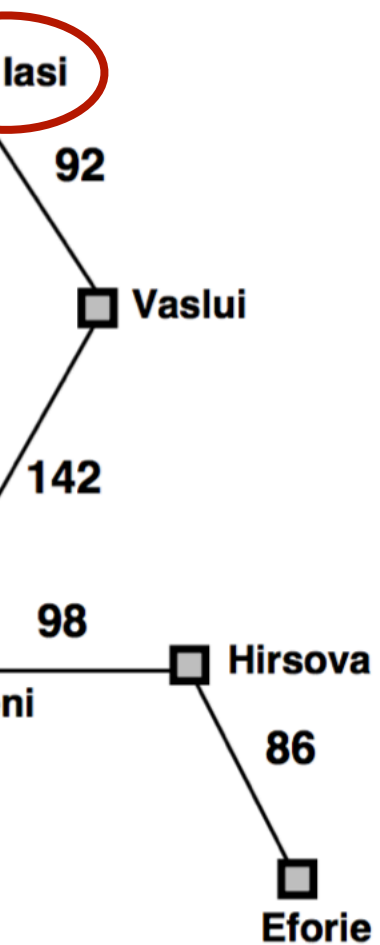
Ricerca Best-First Greedy, un esempio

Oltre a non essere ottimo, l'algoritmo non è neanche completo. Consideriamo il problema di andare da Iasi a Faragas...



Ricerca Best-First Greedy, un esempio

Oltre a non essere ottimo, l'algoritmo non è neanche completo.
Consideriamo il problema di andare da Iasi a Faragas...

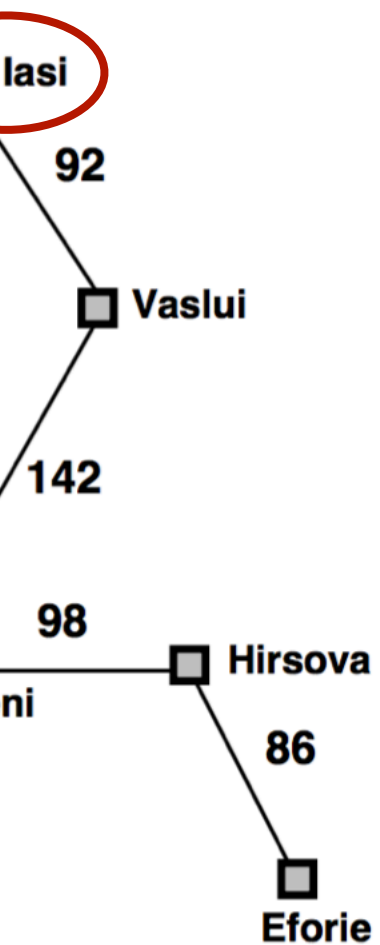


Complessità temporale: Nel caso pessimo, l'algoritmo ha complessità $O(b^m)$, dove m è la profondità massima dello spazio di ricerca. Tuttavia, con una buona euristica, è possibile ridurre drasticamente tale complessità.

Algoritmi di Ricerca Informata

Ricerca Best-First Greedy, un esempio

Oltre a non essere ottimo, l'algoritmo non è neanche completo.
Consideriamo il problema di andare da Iasi a Faragas...



Complessità temporale: Nel caso pessimo, l'algoritmo ha complessità $O(b^m)$, dove m è la profondità massima dello spazio di ricerca. Tuttavia, con una buona euristica, è possibile ridurre drasticamente tale complessità.

Complessità spaziale: Questa sarà nuovamente pari a $O(b^m)$, poiché l'algoritmo conserva tutti i nodi in memoria.

Ricerca A*

La forma più diffusa di ricerca best-first è la ricerca A*, la quale viene eseguita combinando $g(n)$, ovvero il costo per raggiungere il nodo, con $h(n)$, ovvero il cost dal nodo n all'obiettivo. In pratica, $f(n) = g(n) + h(n)$, con $h(n) > 0$ e $h(goal) = 0$.

Ricerca A*

La forma più diffusa di ricerca best-first è la ricerca A*, la quale viene eseguita combinando $g(n)$, ovvero il costo per raggiungere il nodo, con $h(n)$, ovvero il cost dal nodo n all'obiettivo. In pratica, $f(n) = g(n) + h(n)$, con $h(n) > 0$ e $h(goal) = 0$.

Siccome $g(n)$ fornisce il costo del cammino dal nodo iniziale al nodo n ed $f(n)$ rappresenta il costo stimato del cammino più conveniente da n all'obiettivo, allora possiamo riformulare $f(n)$ come:

$f(n)$ = costo stimato della soluzione più conveniente che passa per n .

Algoritmi di Ricerca Informata

Ricerca A*

La forma più diffusa di ricerca best-first è la ricerca A*, la quale viene eseguita combinando $g(n)$, ovvero il costo per raggiungere il nodo, con $h(n)$, ovvero il cost dal nodo n all'obiettivo. In pratica, $f(n) = g(n) + h(n)$, con $h(n) > 0$ e $h(goal) = 0$.

Siccome $g(n)$ fornisce il costo del cammino dal nodo iniziale al nodo n ed $f(n)$ rappresenta il costo stimato del cammino più conveniente da n all'obiettivo, allora possiamo riformulare $f(n)$ come:

$f(n)$ = costo stimato della soluzione più conveniente che passa per n .

Se siamo alla ricerca della soluzione meno costosa, allora è ragionevole provare per primo il nodo con valore più basso di $f(n)$.

Algoritmi di Ricerca Informata

Ricerca A*

La forma più diffusa di ricerca best-first è la ricerca A*, la quale viene eseguita combinando $g(n)$, ovvero il costo per raggiungere il nodo, con $h(n)$, ovvero il cost dal nodo n all'obiettivo. In pratica, $f(n) = g(n) + h(n)$, con $h(n) > 0$ e $h(goal) = 0$.

Siccome $g(n)$ fornisce il costo del cammino dal nodo iniziale al nodo n ed $f(n)$ rappresenta il costo stimato del cammino più conveniente da n all'obiettivo, allora possiamo riformulare $f(n)$ come:

$f(n)$ = costo stimato della soluzione più conveniente che passa per n .

Se siamo alla ricerca della soluzione meno costosa, allora è ragionevole provare per primo il nodo con valore più basso di $f(n)$.

Da un punto di vista pratico, l'algoritmo è implementato nella stessa maniera della ricerca a costo uniforme: l'unica differenza è che la coda a priorità è ordinata in base ad $f(n) = g(n) + h(n)$ piuttosto che $f(n) = g(n)$.

Algoritmi di Ricerca Informata

Ricerca A*

La forma più diffusa di ricerca best-first è la ricerca A*, la quale viene eseguita combinando $g(n)$, ovvero il costo per raggiungere il nodo, con $h(n)$, ovvero il costo dal nodo n all'obiettivo. In pratica, $f(n) = g(n) + h(n)$, con $h(n) > 0$ e $h(goal) = 0$.

Siccome $g(n)$ fornisce il costo del cammino dal nodo iniziale al nodo n ed $f(n)$ rappresenta il costo stimato del cammino più conveniente da n all'obiettivo, allora possiamo riformulare $f(n)$ come:

$f(n)$ = costo stimato della soluzione più conveniente che passa per n .

Se siamo alla ricerca della soluzione meno costosa, allora è ragionevole provare per primo il nodo con valore più basso di $f(n)$.

Da un punto di vista pratico, l'algoritmo è implementato nella stessa maniera della ricerca a costo uniforme: l'unica differenza è che la coda a priorità è ordinata in base ad $f(n) = g(n) + h(n)$ piuttosto che $f(n) = g(n)$.

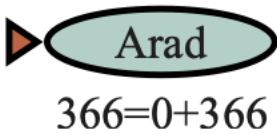
Un paio di casi speciali:

Se $h(n) = 0$, allora $f(n) = g(n) \rightarrow$ si ha una Ricerca Uniforme;

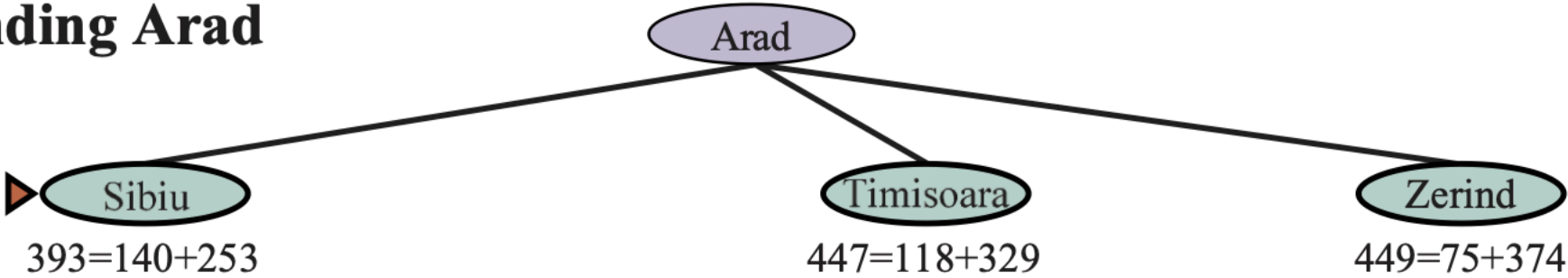
Se $g(n) = 0$, allora $f(n) = h(n) \rightarrow$ si ha una ricerca Best First Greedy.

Ricerca A*, un esempio

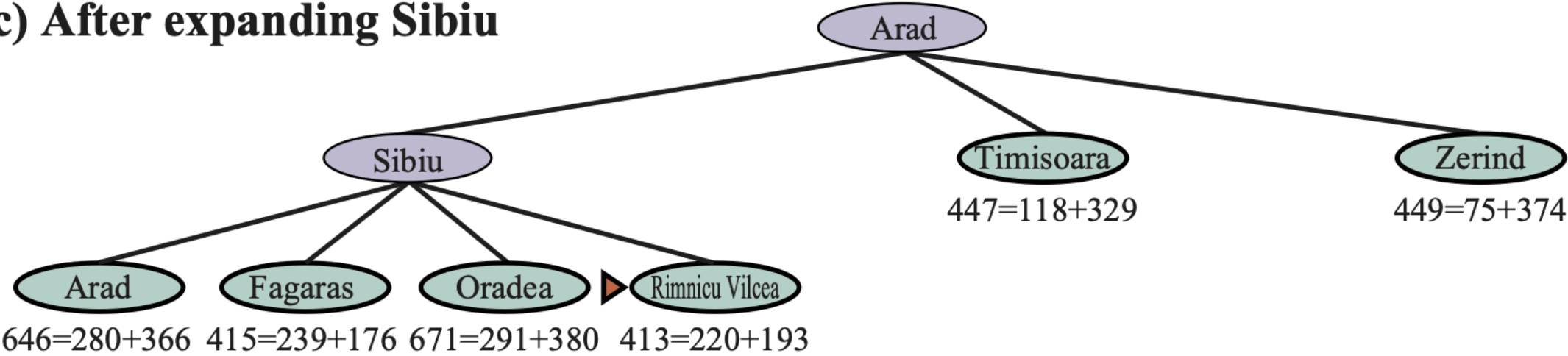
(a) The initial state



(b) After expanding Arad



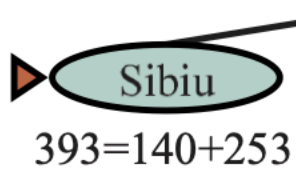
(c) After expanding Sibiu



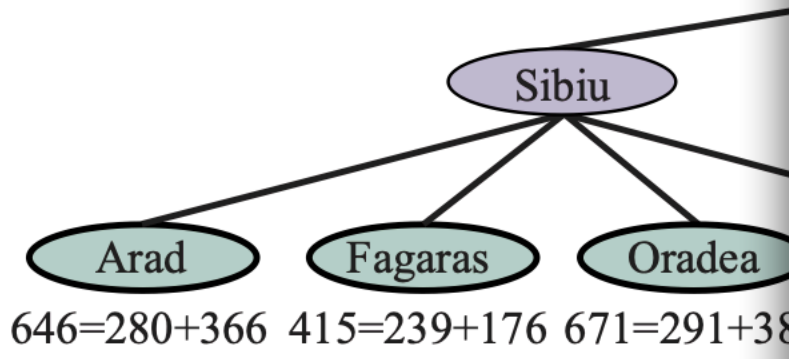
Ricerca A*, un esempio

(a) The initial state

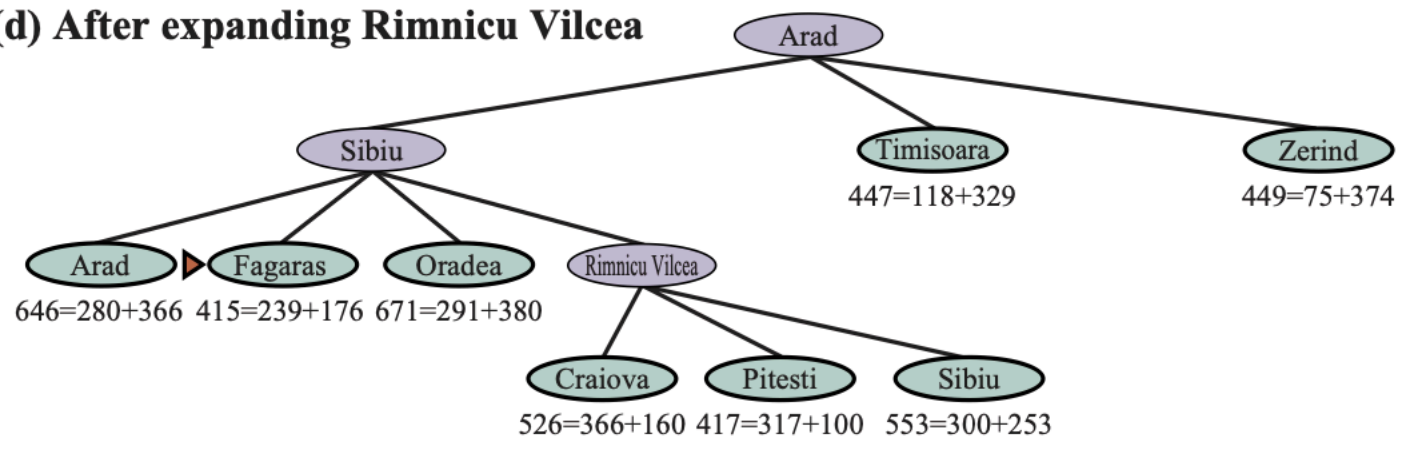
(b) After expanding Arad



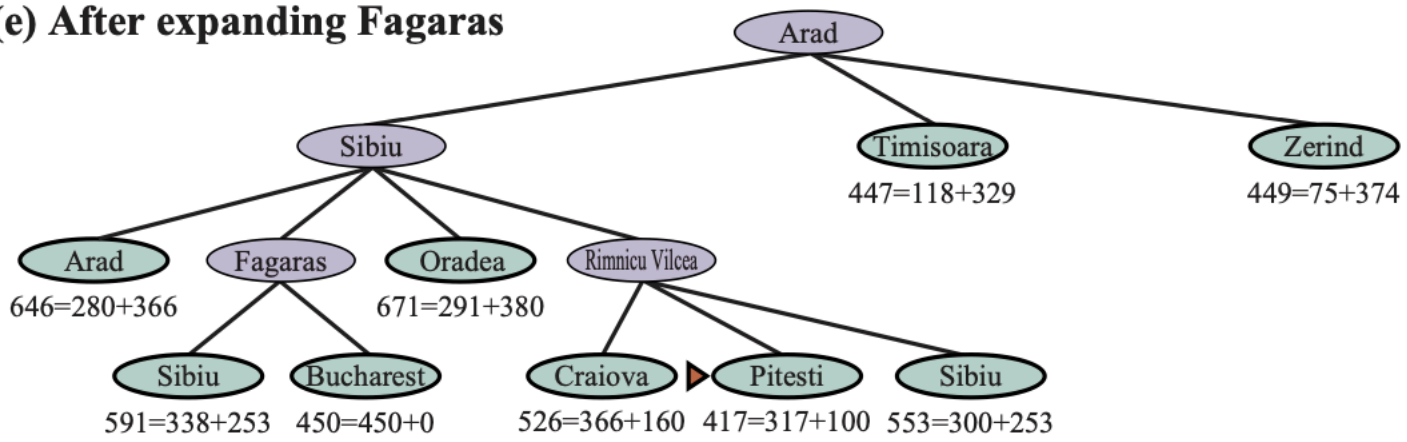
(c) After expanding Sibiu



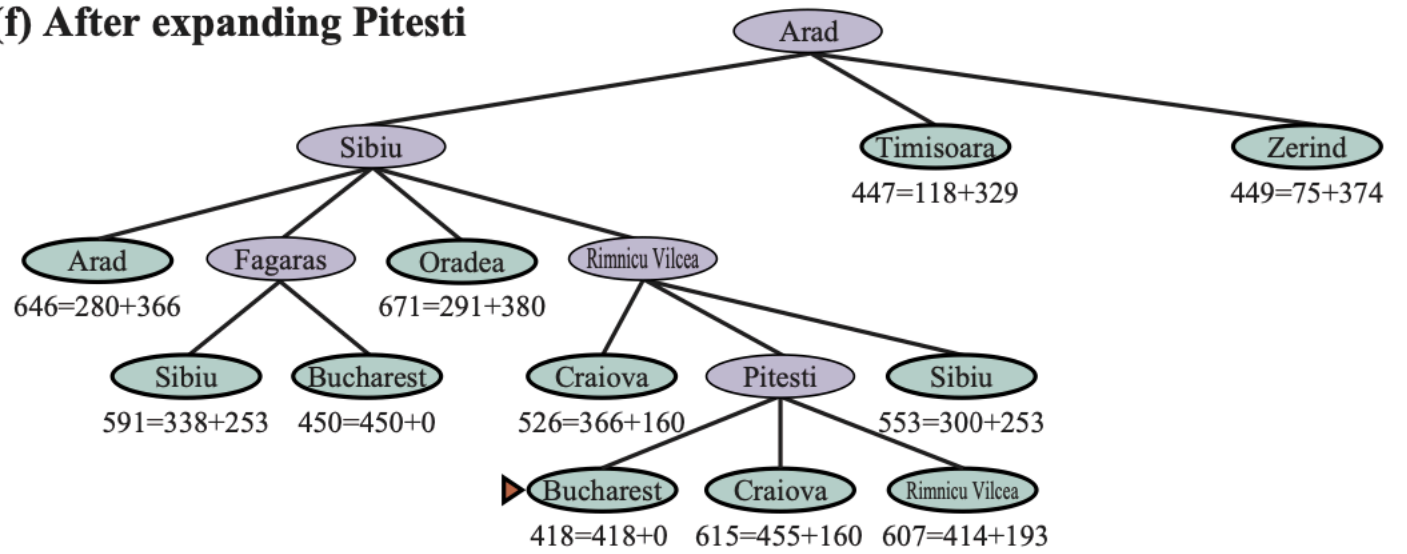
(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



(f) After expanding Pitesti



Ricerca A*, un esempio

Ottimalità: L'ottimalità di A* dipende da due fattori.

Zerind

449=75+374

Zerind

449=75+374

Zerind

449=75+374

Ricerca A*, un esempio

Ottimalità: L'ottimalità di A* dipende da due fattori.

La prima è che $h(n)$ sia un'**euristica ammissibile**, ovvero che non sbagli mai per eccesso la stima del costo per arrivare all'obiettivo.

Zerind

449=75+374

Zerind

449=75+374

Zerind

449=75+374

Ricerca A*, un esempio

Ottimalità: L'ottimalità di A* dipende da due fattori.

La prima è che $h(n)$ sia un'**euristica ammissibile**, ovvero che non sbagli mai per eccesso la stima del costo per arrivare all'obiettivo.

La seconda è che $h(n)$ sia un'**euristica consistente**. Per ogni nodo n e ogni successore n' di n generato da un'azione a , il costo stimato per raggiungere l'obiettivo partendo da n non è superiore al costo di passo per arrivare a n' sommato al costo stimato per andare da lì all'obiettivo.

In altri termini: $h(n) \leq c(n, a, n') + h(n')$

Ricerca A*, un esempio

Ottimalità: L'ottimalità di A* dipende da due fattori.

La prima è che $h(n)$ sia un'**euristica ammissibile**, ovvero che non sbagli mai per eccesso la stima del costo per arrivare all'obiettivo.

La seconda è che $h(n)$ sia un'**euristica consistente**. Per ogni nodo n e ogni successore n' di n generato da un'azione a , il costo stimato per raggiungere l'obiettivo partendo da n non è superiore al costo di passo per arrivare a n' sommato al costo stimato per andare da lì all'obiettivo.

In altri termini: $h(n) \leq c(n, a, n') + h(n')$

Questa è una forma di disuguaglianza triangolare - ogni lato di un triangolo non può mai essere più lungo della somma degli altri due.

Ricerca A*, un esempio

Ottimalità: L'ottimalità di A* dipende da due fattori.

La prima è che $h(n)$ sia un'**euristica ammissibile**, ovvero che non sbagli mai per eccesso la stima del costo per arrivare all'obiettivo.

La seconda è che $h(n)$ sia un'**euristica consistente**. Per ogni nodo n e ogni successore n' di n generato da un'azione a , il costo stimato per raggiungere l'obiettivo partendo da n non è superiore al costo di passo per arrivare a n' sommato al costo stimato per andare da lì all'obiettivo.

In altri termini: $h(n) \leq c(n, a, n') + h(n')$

Questa è una forma di disuguaglianza triangolare - ogni lato di un triangolo non può mai essere più lungo della somma degli altri due.

La dimostrazione dell'ottimalità di A* è sul libro di testo. Gli interessati possono approfondirla.

Ricerca A*, un esempio

Completezza: Questa richiede l'esistenza di un numero finito di nodi di costo minore o uguale a C^* (la soluzione ottima), una condizione vera se tutti i costi dei passi superano un valore finito ϵ e se b è finito.

Zerind

449=75+374

Zerind

449=75+374

Zerind

449=75+374

Ricerca A*, un esempio

Completezza: Questa richiede l'esistenza di un numero finito di nodi di costo minore o uguale a C^* (la soluzione ottima), una condizione vera se tutti i costi dei passi superano un valore finito ϵ e se b è finito.

Complessità temporale: L'algoritmo è ottimamente efficiente. A parità di euristica, nessun altro algoritmo espande meno nodi (senza rinunciare all'ottimalità). I dettagli dell'analisi vanno oltre lo scopo di questo corso, tuttavia possiamo dire che l'algoritmo ha complessità $O(b^\epsilon)$, il che lo rende più efficiente degli altri algoritmi best-first.

Ricerca A*, un esempio

Completezza: Questa richiede l'esistenza di un numero finito di nodi di costo minore o uguale a C^* (la soluzione ottima), una condizione vera se tutti i costi dei passi superano un valore finito ϵ e se b è finito.

Complessità temporale: L'algoritmo è ottimamente efficiente. A parità di euristica, nessun altro algoritmo espande meno nodi (senza rinunciare all'ottimalità). I dettagli dell'analisi vanno oltre lo scopo di questo corso, tuttavia possiamo dire che l'algoritmo ha complessità $O(b^\epsilon)$, il che lo rende più efficiente degli altri algoritmi best-first.

Complessità spaziale: L'algoritmo tiene in memoria tutti i nodi generati, quindi sarà $O(b^m)$.

Ricerca A*, un esempio

Completezza: Questa richiede l'esistenza di un numero finito di nodi di costo minore o uguale a C^* (la soluzione ottima), una condizione vera se tutti i costi dei passi superano un valore finito ϵ e se b è finito.

Complessità temporale: L'algoritmo è ottimamente efficiente. A parità di euristica, nessun altro algoritmo espande meno nodi (senza rinunciare all'ottimalità). I dettagli dell'analisi vanno oltre lo scopo di questo corso, tuttavia possiamo dire che l'algoritmo ha complessità $O(b^\epsilon)$, il che lo rende più efficiente degli altri algoritmi best-first.

Complessità spaziale: L'algoritmo tiene in memoria tutti i nodi generati, quindi sarà $O(b^m)$.

Quindi, con l'algoritmo A* riusciamo a generare soluzioni con una complessità temporale migliore, non riuscendo tuttavia a migliorare quella spaziale... Possiamo, in qualche modo, lavorare su questo aspetto per ottimizzare gli algoritmi visti fino adesso.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

La Beam Search è, quindi, a tutti gli effetti una soluzione *greedy* alla risoluzione dei problemi, poiché espanderà sempre il nodo che ritiene essere più utile al raggiungimento della soluzione.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

La Beam Search è, quindi, a tutti gli effetti una soluzione *greedy* alla risoluzione dei problemi, poiché espanderà sempre il nodo che ritiene essere più utile al raggiungimento della soluzione.

Da un punto di vista pratico, ad ogni livello dell'albero di ricerca l'algoritmo genera tutti i successori degli stati a quel livello, ordinandoli in maniera decrescente in base all'euristica scelta. Conserverà poi solo i primi k di ogni livello, che saranno poi espansi.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

La Beam Search è, quindi, a tutti gli effetti una soluzione *greedy* alla risoluzione dei problemi, poiché espanderà sempre il nodo che ritiene essere più utile al raggiungimento della soluzione.

Da un punto di vista pratico, ad ogni livello dell'albero di ricerca l'algoritmo genera tutti i successori degli stati a quel livello, ordinandoli in maniera decrescente in base all'euristica scelta. Conserverà poi solo i primi k di ogni livello, che saranno poi espansi.

Chiaramente, maggiore sarà il raggio e minore sarà il numero di nodi potati. L'algoritmo restituirà la prima soluzione identificata.

Migliorare l'occupazione di memoria, la Beam Search

La Beam Search è una variante della ricerca best-first che non conserva in memoria tutti i nodi generati, ma tiene ad ogni passo solo i k nodi più promettenti, con k definito come *ampiezza del raggio*.

L'ampiezza del raggio è definita in base ad una euristica (ad esempio, $h(n)$ potrebbe rappresentare l'euristica); solo i nodi nel raggio vengono conservate e considerate come candidati per la soluzione.

La Beam Search è, quindi, a tutti gli effetti una soluzione *greedy* alla risoluzione dei problemi, poiché espanderà sempre il nodo che ritiene essere più utile al raggiungimento della soluzione.

Da un punto di vista pratico, ad ogni livello dell'albero di ricerca l'algoritmo genera tutti i successori degli stati a quel livello, ordinandoli in maniera decrescente in base all'euristica scelta. Conserverà poi solo i primi k di ogni livello, che saranno poi espansi.

Chiaramente, maggiore sarà il raggio e minore sarà il numero di nodi potati. L'algoritmo restituirà la prima soluzione identificata.

Sebbene migliori l'occupazione di memoria (in questo caso, la complessità spaziale sarà pari ad $O(kd)$), l'algoritmo *non è completo né ottimale*.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità limitata

Opera in maniera simile alla strategia di ricerca in profondità ma impone un limite di profondità per evitare che l'algoritmo non termini.

Ricerca in profondità limitata: Con questa strategia, un nodo viene espanso solo se la lunghezza del cammino corrispondente è minore rispetto al massimo stabilito.

Da un lato, l'algoritmo evita la non terminazione; dall'altro, però, introduce una nuova fonte di incompletezza nel caso in cui la soluzione si trovi ad una profondità maggiore di quella stabilita.

Rispetto ai precedenti algoritmi, questo tipo di ricerca può terminare in due modi diversi: o per via di un fallimento (ovvero, nessuna soluzione) o tramite il valore speciale *taglio* (ovvero, nessuna soluzione entro il limite stabilito).

Il problema di questa strategia consiste nel selezionare una soglia tale da poter garantirne il successo. Talvolta, questa soglia deriva dall'esperienza del designer o da dati empirici; altre volte, invece, non è possibile definirla.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca in profondità

Opera in maniera simile alla strategia in ampiezza per evitare che l'algoritmo esplori nodi inutilmente.

Ricerca in profondità limitata: Con un limite sulla lunghezza del cammino corrisponde a una profondità limitata.

Da un lato, l'algoritmo evita la non terminazione in caso di infinitezza del problema; dall'altro, evita la fonte di incompletezza nel caso in cui il problema non è risolubile.

Rispetto ai precedenti algoritmi, questa strategia è diversa: o per via di un fallimento (ovvero, nessuna soluzione) o per via di un taglio (ovvero, nessuna soluzione oltre un certo punto).

Il problema di questa strategia consiste nel garantire il successo. Talvolta, questa strategia è data empiricamente; altre volte, invece, non è.

Algoritmi di Ricerca Non Informata

Strategie di ricerca non informata - Ricerca ad approfondimento iterativo

Questo tipo di ricerca ha l'obiettivo di identificare un limite ideale per la ricerca a profondità limitata, tuttavia non compromettendo costo, completezza e ottimalità.

Ricerca ad approfondimento iterativo: Con questa strategia, il limite viene incrementato progressivamente, finché un nodo obiettivo non è identificato.

```
function RICERCA-APPROFONDIMENTO-ITERATIVO(problema)
    returns una soluzione o un fallimento

    for profondità = 0 to ∞ do
        risultato ← RICERCA-PROFONDITÀ-LIMITATA(problema, profondità)
        if risultato != fallimento then return risultato
```

Oltre a migliorare la ricerca in profondità, la ricerca ad approfondimento iterativo è analoga a quella in ampiezza, poiché ad ogni iterazione esplora completamente un livello di nodi prima di prendere in considerazione il successivo.

In generale, questo tipo di ricerca è il metodo di ricerca non informata da preferire quando lo spazio di ricerca è grande e la profondità della soluzione non è nota.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

La differenza principale tra IDA* e l'approfondimento iterativo standard sta nel valore del taglio, che non è più basato sulla profondità ma sul costo f , ovvero su $g+h$.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

La differenza principale tra IDA* e l'approfondimento iterativo standard sta nel valore del taglio, che non è più basato sulla profondità ma sul costo f , ovvero su $g+h$.

Ad ogni iterazione, il nuovo valore di taglio è l' f -costo minimo tra quelli di tutti i nodi che hanno superato il valore di taglio nell'iterazione precedente.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

La differenza principale tra IDA* e l'approfondimento iterativo standard sta nel valore del taglio, che non è più basato sulla profondità ma sul costo f , ovvero su $g+h$.

Ad ogni iterazione, il nuovo valore di taglio è l' f -costo minimo tra quelli di tutti i nodi che hanno superato il valore di taglio nell'iterazione precedente.

Essendo una estensione di A*, ne mantiene le proprietà di completezza e ottimalità, a patto che sussistano le condizioni di ammissibilità e consistenza.

Migliorare l'occupazione di memoria, la Iterative Deepening A* (IDA*)

È possibile adottare l'idea dell'approfondimento iterativo al contesto della ricerca euristica: questo dà vita all'algoritmo IDA*.

La differenza principale tra IDA* e l'approfondimento iterativo standard sta nel valore del taglio, che non è più basato sulla profondità ma sul costo f , ovvero su $g+h$.

Ad ogni iterazione, il nuovo valore di taglio è l' f -costo minimo tra quelli di tutti i nodi che hanno superato il valore di taglio nell'iterazione precedente.

Essendo una estensione di A*, ne mantiene le proprietà di completezza e ottimalità, a patto che sussistano le condizioni di ammissibilità e consistenza.

Il vero vantaggio si ottiene in termini di occupazione di memoria, poiché l'algoritmo mantiene le proprietà dell'algoritmo di ricerca ad approfondimento iterativo standard: la ricerca dovrà memorizzare un solo cammino dalla radice ad un nodo foglia, insieme ai rimanenti nodi fratelli non espansi per ciascun nodo sul cammino. Una volta che un nodo è stato espanso, può essere rimosso dalla memoria non appena tutti i suoi discendenti sono stati esplorati completamente. Per cui, la complessità sarà di $O(bd)$.

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

Questa rappresenta una variante della ricerca best-first classica, usando tuttavia solo uno spazio lineare.

Ad ogni iterazione, l'algoritmo tiene traccia del miglior percorso alternativo. Invece di fare backtracking in caso di fallimento, *interrompe l'esplorazione quando trova un nodo meno promettente* (definito sulla base di f).

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

```
function RICERCA-BEST-FIRST-RICORSIVA(problema)
    returns una soluzione o il fallimento

    returns RBFS(problema, CREA-NODO(problema.STATO-INIZIALE),  $\infty$ )

function RBFS(problema, nodo, f_limite)
    returns una soluzione o il fallimento e un nuovo limite all'f-costo

    if problema.TEST-OBIETTIVO(nodo.STATO) then returns SOLUZIONE(nodo)
    successori  $\leftarrow$  [ ]

    for each azione in problema.AZIONI(nodo.STATO) do
        aggiungi NODO.FIGLIO(problema, nodo, azione) a successori

    if successori è vuoto then returns fallimento,  $\infty$ 

    for each s in successori do
        s.f  $\leftarrow$  max(s.g + s.h, nodo.f)
    loop do
        migliore  $\leftarrow$  il nodo con f-valore minimo in successori
        if migliore.f > f_limite then returns fallimento, migliore.f
        alternativa  $\leftarrow$  il secondo nodo con f-valore minimo in successori
        risultato, migliore.f  $\leftarrow$  RBFS(problema, migliore, min(f_limite, alternativa))
        if risultato  $\neq$  fallimento then return fallimento
```

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

```
function RICERCA-BEST-FIRST-RICORSIVA(problema)
    returns una soluzione o il fallimento
```

```
    returns RBFS(problema, CREA-NODO(problema.STATO-INIZIALE),  $\infty$ )
```

```
function RBFS(problema, nodo, f_limite)
    returns una soluzione o il fallimento e un nuovo limite all'f-costo
```

```
    if soluzione = SOLUZIONE(nodo)
```

```
    successori ← /* La struttura dell'implementazione è simile a quella della ricerca
    ricorsiva in profondità */
```

```
    for azione in azioni(nodo)
        aggiungi NODO.FIGLIO(problema, nodo, azione) a successori
```

```
    if successori è vuoto then returns fallimento,  $\infty$ 
```

```
    for each s in successori do
```

```
        s.f ← max(s.g + s.h, nodo.f)
```

```
    loop do
```

```
        migliore ← il nodo con f-valore minimo in successori
```

```
        if migliore.f > f_limite then returns fallimento, migliore.f
```

```
        alternativa ← il secondo nodo con f-valore minimo in successori
```

```
        risultato, migliore.f ← RBFS(problema, migliore, min(f_limite, alternativa))
```

```
        if risultato ≠ fallimento then return fallimento
```

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

```
function RICERCA-BEST-FIRST-RICORSIVA(problema)
    returns una soluzione o il fallimento
```

```
    returns RBFS(problema, CREA-NODO(problema.STATO-INIZIALE),  $\infty$ )
```

```
function RBFS(problema, nodo, f_limite)
    returns una soluzione o il fallimento e un nuovo limite all'f-costo
```

```
if problema.TEST-OBIETTIVO(nodo)
    successori  $\leftarrow$  [ ]
```

```
for each azione in problema.AZIONI
```

```
    aggiungi NODO.FIGLIO(problema, nodo, azione) a successori
```

```
if successori è vuoto then returns fallimento,  $\infty$ 
```

```
for each s in successori do
```

```
     $s.f \leftarrow \max(s.g + s.h, \text{nodo}.f)$ 
```

```
loop do
```

```
    migliore  $\leftarrow$  il nodo con f-valore minimo in successori
```

```
    if migliore.f > f_limite then returns fallimento, migliore.f
```

```
    alternativa  $\leftarrow$  il secondo nodo con f-valore minimo in successori
```

```
    risultato, migliore.f  $\leftarrow$  RBFS(problema, migliore, min(f_limite, alternativa))
```

```
    if risultato  $\neq$  fallimento then return fallimento
```

/* Ma invece di continuare a seguire indefinitamente il cammino corrente, si utilizza la variabile f_limite per tenere traccia dell'f-valore del miglior cammino alternativo */

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

```
function RICERCA-BEST-FIRST-RICORSIVA(problema)
    returns una soluzione o il fallimento
```

```
    returns RBFS(problema, CREA-NODO(problema.STATO-INIZIALE),  $\infty$ )
```

```
function RBFS(problema, nodo, f_limite)
    returns una soluzione o il fallimento e un nuovo limite all'f-costo
```

```
    if problema.TEST-OBIETTIVO(nodo.STATO) then returns SOLUZIONE(nodo)
    successori  $\leftarrow$  [ ]
```

```
    for each azione in problema.AZIONI(nodo.STATO) do
        aggiungi NODO.FIGLIO(problema, nodo, azione) a successori
```

```
    if succ
```

```
    for ead
```

```
        s.f
```

```
    loop do
```

```
        migliore  $\leftarrow$  il nodo con f-valore minimo in successori
```

```
        if migliore.f > f_limite then returns fallimento, migliore.f
```

```
        alternativa  $\leftarrow$  il secondo nodo con f-valore minimo in successori
```

```
        risultato, migliore.f  $\leftarrow$  RBFS(problema, migliore, min(f_limite, alternativa))
```

```
        if risultato  $\neq$  fallimento then return fallimento
```

/* Se il nodo corrente supera il limite, la ricorsione torna indietro
così che l'algoritmo considererà il cammino alternativo */

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

Questa rappresenta una variante della ricerca best-first classica, usando tuttavia solo uno spazio lineare.

Ad ogni iterazione, l'algoritmo tiene traccia del miglior percorso alternativo. Invece di fare backtracking in caso di fallimento, *interrompe l'esplorazione quando trova un nodo meno promettente* (definito sulla base di f).

In primo luogo, la ricerca Best-First ricorsiva è ottimale se la funzione euristica $h(n)$ è ammissibile (non sbaglia mai per eccesso la stima del costo per arrivare all'obiettivo).

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

Questa rappresenta una variante della ricerca best-first classica, usando tuttavia solo uno spazio lineare.

Ad ogni iterazione, l'algoritmo tiene traccia del miglior percorso alternativo. Invece di fare backtracking in caso di fallimento, *interrompe l'esplorazione quando trova un nodo meno promettente* (definito sulla base di f).

In primo luogo, la ricerca Best-First ricorsiva è ottimale se la funzione euristica $h(n)$ è ammissibile (non sbaglia mai per eccesso la stima del costo per arrivare all'obiettivo).

La complessità spaziale è lineare nella profondità della più profonda soluzione ottima — $O(bd)$ — essendo quindi notevolmente migliore rispetto alle alternative precedenti, che presentavano una complessità esponenziale.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

Questa rappresenta una variante della ricerca best-first classica, usando tuttavia solo uno spazio lineare.

Ad ogni iterazione, l'algoritmo tiene traccia del miglior percorso alternativo. Invece di fare backtracking in caso di fallimento, *interrompe l'esplorazione quando trova un nodo meno promettente* (definito sulla base di f).

In primo luogo, la ricerca Best-First ricorsiva è ottimale se la funzione euristica $h(n)$ è ammissibile (non sbaglia mai per eccesso la stima del costo per arrivare all'obiettivo).

La complessità spaziale è lineare nella profondità della più profonda soluzione ottima — $O(bd)$ — essendo quindi notevolmente migliore rispetto alle alternative precedenti, che presentavano una complessità esponenziale.

La complessità temporale è però difficile da definire. Questa dipende sia dalla funzione euristica sia dalla frequenza dei cambiamenti del cammino ottimo durante l'espansione dei nodi. Nel caso pessimo, però, sarà esponenziale.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Best-First ricorsiva

Questa rappresenta una variante della ricerca best-first classica, usando tuttavia solo uno spazio lineare.

Ad ogni iterazione, l'algoritmo tiene traccia del miglior percorso alternativo. Invece di fare backtracking in caso di fallimento, *interrompe l'esplorazione quando trova un nodo meno promettente* (definito sulla base di f).

In primo luogo, la ricerca Best-First ricorsiva è ottimale se la funzione euristica $h(n)$ è ammissibile (non sbaglia mai per eccesso la stima del costo per arrivare all'obiettivo).

La complessità spaziale è lineare nella profondità della più profonda soluzione ottima — $O(bd)$ — essendo quindi notevolmente migliore rispetto alle alternative precedenti, che presentavano una complessità esponenziale.

La complessità temporale è però difficile da definire. Questa dipende sia dalla funzione euristica sia dalla frequenza dei cambiamenti del cammino ottimo durante l'espansione dei nodi. Nel caso pessimo, però, sarà esponenziale.

Il problema della ricerca Best-First ricorsiva, così come di IDA*, consiste nel fatto che utilizzano **troppa poca memoria**. Tra un'iterazione ed un'altra, IDA* ricorda solo il limite corrente all' f -costo, mentre RBFS usa solo uno spazio lineare. Anche se fosse disponibile più memoria, i due algoritmi non riuscirebbero ad usarla. Entrambi gli algoritmi dimenticano la maggior parte di ciò che fanno, **rischiando di espandere più e più volte gli stessi stati già visitati in precedenza**.

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Quando questo accade, allora l'algoritmo libera il nodo peggiore, ovvero quello con l' f -valore più alto. Come RBFS, però, memorizza nel nodo padre il valore del nodo dimenticato.

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Quando questo accade, allora l'algoritmo libera il nodo peggiore, ovvero quello con l'*f*-valore più alto. Come RBFS, però, memorizza nel nodo padre il valore del nodo dimenticato.

Questo meccanismo consente all'algoritmo di tenere traccia della radice di un sotto-albero dimenticato, avendo quindi a disposizione l'informazione sul cammino migliore di quel sotto-albero —> SMA* rigenera un sotto-albero dimenticato solo quando tutti gli altri cammini promettono di comportarsi peggio di quello.

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Quando questo accade, allora l'algoritmo libera il nodo peggiore, ovvero quello con l' f -valore più alto. Come RBFS, però, memorizza nel nodo padre il valore del nodo dimenticato.

Questo meccanismo consente all'algoritmo di tenere traccia della radice di un sotto-albero dimenticato, avendo quindi a disposizione l'informazione sul cammino migliore di quel sotto-albero —> SMA* rigenera un sotto-albero dimenticato solo quando tutti gli altri cammini promettono di comportarsi peggio di quello.

SMA* è completo se la soluzione è raggiungibile, ovvero se d - la profondità del nodo obiettivo più vicino alla radice - è inferiore alla dimensione della memoria espressa in nodi.

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Quando questo accade, allora l'algoritmo libera il nodo peggiore, ovvero quello con l' f -valore più alto. Come RBFS, però, memorizza nel nodo padre il valore del nodo dimenticato.

Questo meccanismo consente all'algoritmo di tenere traccia della radice di un sotto-albero dimenticato, avendo quindi a disposizione l'informazione sul cammino migliore di quel sotto-albero —> SMA* rigenera un sotto-albero dimenticato solo quando tutti gli altri cammini promettono di comportarsi peggio di quello.

SMA* è completo se la soluzione è raggiungibile, ovvero se d - la profondità del nodo obiettivo più vicino alla radice - è inferiore alla dimensione della memoria espressa in nodi.

SMA* è ottimale se c'è una soluzione ottima raggiungibile.

Algoritmi di Ricerca Informata

Migliorare l'occupazione di memoria, la ricerca Simplified Memory Bounded A* (SMA*)

L'idea alla base è quella di utilizzare meglio la memoria disponibile. Molto semplicemente, SMA* procede come A* fino all'esaurimento della memoria disponibile.

Quando questo accade, allora l'algoritmo libera il nodo peggiore, ovvero quello con l' f -valore più alto. Come RBFS, però, memorizza nel nodo padre il valore del nodo dimenticato.

Questo meccanismo consente all'algoritmo di tenere traccia della radice di un sotto-albero dimenticato, avendo quindi a disposizione l'informazione sul cammino migliore di quel sotto-albero —> SMA* rigenera un sotto-albero dimenticato solo quando tutti gli altri cammini promettono di comportarsi peggio di quello.

SMA* è completo se la soluzione è raggiungibile, ovvero se d - la profondità del nodo obiettivo più vicino alla radice - è inferiore alla dimensione della memoria espressa in nodi.

SMA* è ottimale se c'è una soluzione ottima raggiungibile.

Il problema di questo algoritmo consiste nel fatto che, per problemi difficili, la ricerca sarà spesso obbligata a passare continuamente dall'uno all'altro tra molti cammini candidati di cui sarà possibile memorizzare solo un piccolo sottoinsieme. Quando questo accade, il problema potrebbe diventare intrattabile —> la complessità temporale potrebbe esplodere, rendendo il problema non risolvibile nella pratica.

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Algoritmi di Ricerca Informata

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Migliorando anche di poco un'euristica si riesce ad esplorare uno spazio degli stati molto più vasto.

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Migliorando anche di poco un'euristica si riesce ad esplorare uno spazio degli stati molto più vasto.

Per la risoluzione di problemi, spesso dovrete “inventare” delle euristiche:

- *Rilassamento dei vincoli*: Pensare ad un problema con meno vincoli;

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Migliorando anche di poco un'euristica si riesce ad esplorare uno spazio degli stati molto più vasto.

Per la risoluzione di problemi, spesso dovrete “inventare” delle euristiche:

- *Rilassamento dei vincoli*: Pensare ad un problema con meno vincoli;
- *Massimizzazione di euristiche*: Se si hanno una serie di euristiche $h_1, h_2, \dots, h_k$, senza dominanza tra queste allora potrebbe convenire massimizzare la funzione: $h(n) = \max(h_1(n), h_2(n), \dots, h_k(n))$.

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Migliorando anche di poco un'euristica si riesce ad esplorare uno spazio degli stati molto più vasto.

Per la risoluzione di problemi, spesso dovrete “inventare” delle euristiche:

- *Rilassamento dei vincoli*: Pensare ad un problema con meno vincoli;
- *Massimizzazione di euristiche*: Se si hanno una serie di euristiche $h_1, h_2, \dots, h_k$, senza dominanza tra queste allora potrebbe convenire massimizzare la funzione: $h(n) = \max(h_1(n), h_2(n), \dots, h_k(n))$.
- *Apprendere dall'esperienza*: Usare dati passati per apprendere un'euristica —> utilizzare tecniche di apprendimento per la risoluzione di problemi di ricerca;

Algoritmi di Ricerca Informata e Funzioni Euristiche, la lezione da apprendere

Molti problemi dell'Intelligenza Artificiale sono di complessità esponenziale: tuttavia, c'è esponenziale e esponenziale!

Una buona euristica può migliorare di molto la capacità di esplorazione dello spazio degli stati rispetto alla ricerca cieca.

Migliorando anche di poco un'euristica si riesce ad esplorare uno spazio degli stati molto più vasto.

Per la risoluzione di problemi, spesso dovrete “inventare” delle euristiche:

- *Rilassamento dei vincoli*: Pensare ad un problema con meno vincoli;
- *Massimizzazione di euristiche*: Se si hanno una serie di euristiche $h_1, h_2, \dots, h_k$, senza dominanza tra queste allora potrebbe convenire massimizzare la funzione: $h(n) = \max(h_1(n), h_2(n), \dots, h_k(n))$.
- *Apprendere dall'esperienza*: Usare dati passati per apprendere un'euristica —> utilizzare tecniche di apprendimento per la risoluzione di problemi di ricerca;
- *Combinare euristiche*: Quando diverse caratteristiche influenzano la bontà di uno stato, allora si può usare una combinazione lineare del tipo:
 $h(n) = c_1 \cdot h_1(n) + c_2 \cdot h_2(n) + \dots + c_k \cdot h_k(n)$.



UNIVERSITÀ DEGLI STUDI DI SALERNO
DIPARTIMENTO DI INFORMATICA

Laurea triennale in Informatica

Fondamenti di Intelligenza Artificiale

Lezione 5 - Algoritmi di ricerca informata

